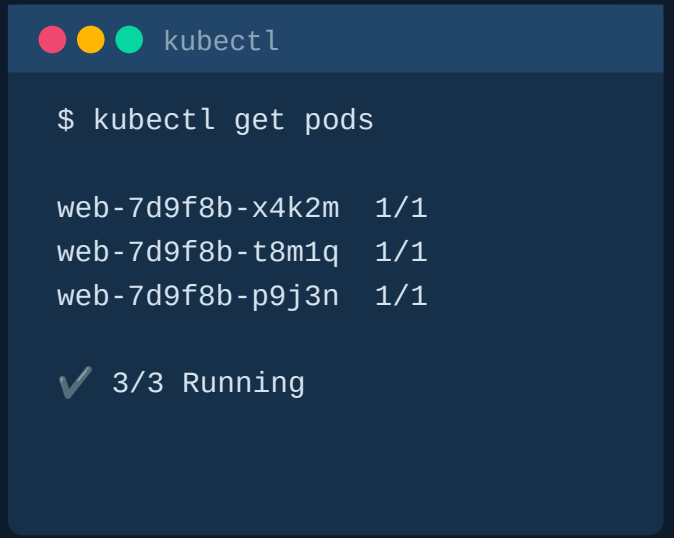
CODERHOUSE · DEVOPS & CLOUD

Sesión en vivo 4

Orquestación efectiva con Kubernetes



Pods · Services · Ingress · Auto-escalado

Formato taller · Dos actividades en grupo · 2 h

Cómo vamos a trabajar hoy

Más taller que clase: la mitad del tiempo van a estar con las manos en el teclado

5'	**Bienvenida**	Verificamos entornos y armamos los grupos
10'	**Repaso y explicación guiada**	Pods, Services, Ingress y auto-escalado
10'	**Demostración en vivo**	Desplegamos juntos y rompemos cosas a propósito
5'	**Pausa activa**	Estiren las piernas
20'	**Actividad 1 en grupos**	Desplegar un Pod y exponerlo con un Service
10'	**Puesta en común**	Qué les salió, qué se les rompió
5'	**Pausa corta**	Recargamos
20'	**Actividad 2 en grupos**	Configurar Ingress y auto-escalado por CPU
15'	**Preguntas y cierre**	Dudas, próximos pasos y tarea

REPASO RÁPIDO · 5 MINUTOS

Antes de arrancar: ¿qué se acuerdan?

Respondan por chat o micrófono. No busquen en Google: quiero saber qué quedó del módulo.

1. ¿Qué es un Pod? ¿Es lo mismo que un contenedor?
2. ¿Para qué sirve un Service si los pods ya tienen IP?
3. ¿Qué diferencia hay entre Docker y Kubernetes?
4. Si borro un pod a mano, ¿qué pasa?

No corrijas todavía: anotá las respuestas y volvé a ellas durante la explicación guiada.

01

EXPLICACIÓN GUIADA · 10 MINUTOS

Los cuatro objetos de hoy

Pod, Service, Ingress y HPA. Son exactamente los que van a usar en las dos actividades.

La idea de la que sale todo lo demás

Vos no das órdenes: describís cómo querés que se vea el mundo

Kubernetes compara permanentemente lo que pediste contra lo que existe, y corrige la diferencia. No ejecuta tu orden y se olvida.

IMPERATIVO · una receta de pasos

"Levantá tres contenedores"

Si lo corro dos veces, tengo seis. Si uno se muere, quedan dos y nadie hace nada: el comando ya terminó.

DECLARATIVO · un estado a sostener

"Quiero que haya tres, siempre"

Si lo aplico dos veces, siguen siendo tres. Si uno se muere, el sistema cuenta dos, ve que falta y lo crea.

Esto responde la pregunta 4 del repaso: si borrás un pod a mano, aparece otro. No porque Kubernetes te lleve la contra, sino porque vos dijiste que querías tres y él te va a dar tres.

Pod y Deployment

El pod es la unidad mínima; el Deployment es el que la mantiene viva

Pod

Uno o más contenedores que viven juntos

- Comparten IP y almacenamiento
- Siempre en el mismo nodo
- Es descartable: nace, vive y muere. No se repara, se reemplaza

Deployment

El que garantiza que siempre haya N pods

- Un pod suelto no tiene quién lo vigile: si muere, se murió
- El Deployment lo repone y además maneja las actualizaciones
- Es lo que vas a escribir el 90 % de las veces

```
kind: Deployment
spec:
  replicas: 3
  selector:
    matchLabels: { app: web }   # ← debe coincidir
  template:
    metadata:
      labels: { app: web }      # ← con esta
```

El error número uno

Que el selector no coincida con las labels del template. El Deployment se crea, NO da error, y no levanta ningún pod. Es un fallo silencioso y lo van a sufrir hoy.

Service — la dirección que no cambia

Responde la pregunta 2 del repaso: los pods tienen IP, pero cambia en cada reinicio

Si tu frontend le pegara directo a la IP de un pod, se rompería en el primer reinicio. El Service te da un nombre fijo que siempre apunta a los pods vivos, y reparte el tráfico entre ellos.

ClusterIP

Solo adentro del cluster

El predeterminado. Para que un servicio le hable a otro.

Lo usás siempre por defecto

NodePort

Abre un puerto en cada nodo

Rango 30000-32767. Útil para probar en Minikube.

Lo vamos a usar en la actividad 1

LoadBalancer

Pide un balanceador a la nube

Lo que usás en producción para exponer a internet.

Fuera de la nube no funciona

```
kind: Service
spec:
  selector: { app: web }     # ← se conecta por LABELS, no por nombre del Deployment
```

Ingress — una sola puerta de entrada

Lo que van a configurar en la actividad 2

EL PROBLEMA

Con NodePort, cada servicio se lleva un puerto raro. Con LoadBalancer, cada servicio se lleva una IP pública que te cobran aparte.

Con diez servicios, tenés diez puertos o diez facturas.

LA SOLUCIÓN

Una única entrada HTTP que enruta por nombre de dominio o por ruta hacia el Service que corresponda.

Una IP, un certificado TLS, y todas las reglas en un solo lugar.

ingress.yaml

```
kind: Ingress
spec:
  rules:
    - host: mi-app.local
      http:
        paths:
          - path: /
            backend:
              service: { name: web-svc, port: 80 }
```

Ojo con esto

El objeto Ingress es solo una lista de reglas. Para que haga algo tiene que haber un Ingress Controller instalado que las lea y las aplique.

```
En Minikube:
minikube addons enable ingress
```

Auto-escalado (HPA)

El otro objeto de la actividad 2: ajusta las réplicas según la carga

hpa.yaml

```
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef:
    kind: Deployment
    name: web          # a quien escala
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - resource: { name: cpu,
        target: { averageUtilization: 50 }}
```

Cómo decide

Mira el uso promedio de CPU de todos los pods. Si supera el 50 %, agrega réplicas. Si baja, las saca. Nunca menos de 2 ni más de 10.

Para bajar espera unos minutos, para evitar prender y apagar todo el tiempo.

Dos requisitos sin los cuales el HPA no funciona, y los dos fallan en silencio

1 · El Deployment tiene que tener resources.requests.cpu definido. El HPA calcula el uso como porcentaje del request: sin denominador, no hay cuenta.

2 · Tiene que estar el metrics-server instalado. En Minikube: minikube addons enable metrics-server

```
$ kubectl get hpa
NAME  REFERENCE       TARGETS        MINPODS MAXPODS REPLICAS
web   Deployment/web  <unknown>/50%  2       10      2    <- falta metrics-server
```

02

DEMOSTRACIÓN EN VIVO · 10 MINUTOS

Lo hacemos juntos

Miren ahora, lo replican en la actividad. No hace falta que me sigan en paralelo.

DEMOSTRACIÓN

Desplegar, romper y ver cómo se repara

terminal 1 — desplegar

```
# 1. Verificar el cluster
kubectl get nodes

# 2. Desplegar
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml

# 3. Ver que levanto
kubectl get pods
kubectl get svc

# 4. Entrar a la app
minikube service web-svc
```

terminal 2 — observar

```
# 5. Dejar esto mirando
kubectl get pods --watch

# 6. En la otra terminal:
kubectl delete pod <nombre>

#    ← aparece uno nuevo solo

# 7. Escalar en vivo
kubectl scale deploy/web \
  --replicas=5

# 8. Diagnosticar
kubectl describe pod <nombre>
```

El momento clave es el paso 6: borrás un pod y aparece otro en tres segundos. Quedate en silencio y dejá que lo vean.

Pausa activa

5 minutos

Al volver: primera actividad en grupos

ACTIVIDAD 1 · 20 MINUTOS

03

Pod y Service en grupos

Grupos de 3 o 4. Desplegar una aplicación y exponerla para poder abrirla en el navegador.

Actividad 1 · La consigna

20 minutos · Grupos de 3 o 4 · Una persona comparte pantalla y las demás dictan

1	**Desplegar**	Crear un Deployment con 3 réplicas de nginx:alpine o de su propia imagen
2	**Verificar**	Confirmar con kubectl get pods que los 3 están en Running
3	**Exponer**	Crear un Service de tipo NodePort que apunte a esos pods
4	**Abrir**	Entrar a la aplicación desde el navegador y ver que responde
5	**Romper**	Borrar un pod a mano y observar qué pasa con kubectl get pods --watch

Para el que termine antes: agregarle un ConfigMap con una variable de entorno y verificarla con kubectl exec.

Actividad 1 · Con qué arrancar

Estos dos archivos están en el material. Copiarlos y completar lo que falta.

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: ___    # <- completar
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: ___   # <- completar
    spec:
      containers:
        - name: web
          image: nginx:alpine
```

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  type: ___        # <- completar
  selector:
    app: ___       # <- completar
  ports:
    - port: 80
      targetPort: 80

# Los comandos:
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
minikube service web-svc
```

Los cuatro huecos no son casuales

Dos de ellos son labels que tienen que coincidir entre los tres lugares: el selector del Deployment, el template del Deployment y el selector del Service. Si no coinciden, no hay error, simplemente no funciona.

Si algo no anda

El comando que resuelve casi todo, y los seis errores que vas a ver hoy

```
kubectl describe pod <nombre>     # ← mirá la seccion Events, al final de la salida
```

`Pending`

No hay nodo con recursos libres, o el cluster no arrancó bien

`ImagePullBackOff`

No encuentra la imagen: revisá el nombre y el tag

`CrashLoopBackOff`

La app arranca y se muere sola. Mirá kubectl logs

`No hay ningún pod`

El selector no coincide con las labels del template

`El Service no responde`

Las labels del Service no coinciden con las de los pods

`ErrImagePull en Minikube`

Si es tu imagen local: minikube image load mi-app:v1

Regla: antes de preguntar, corré describe pod y leé los Events. La respuesta está ahí el 80 % de las veces.

PUESTA EN COMÚN · 10 MINUTOS

¿Cómo les fue?

Dos o tres grupos comparten. Nos interesan tanto los aciertos como lo que se rompió.

¿Qué les costó más?	*Lo más frecuente: el entorno, no el concepto*
¿A alguien no le levantaron los pods?	*Casi siempre es el selector vs. las labels*
¿Qué pasó al borrar el pod?	*Que apareció otro. ¿Por qué? Porque dijeron replicas: 3*
¿Por qué NodePort y no LoadBalancer?	*Porque fuera de la nube no hay quién provea el balanceador*
¿Qué pasaría con replicas: 1?	*Al borrar el pod, habría corte real hasta que levante el nuevo*

La columna verde es para vos, no la leas: dejá que la respuesta salga del grupo.

Pausa corta

5 minutos

Al volver: Ingress y auto-escalado

04

ACTIVIDAD 2 · 20 MINUTOS

Ingress y auto-escalado

Mismos grupos. Ahora exponemos por nombre de dominio y hacemos que escale solo.

Actividad 2 · La consigna

20 minutos · Sobre el mismo despliegue de la actividad anterior

1	**Preparar**	Habilitar los addons de ingress y metrics-server si no lo hicieron en la pausa
2	**Requests**	Agregarle al Deployment resources.requests.cpu — sin esto el HPA no funciona
3	**Ingress**	Crear un Ingress que enrute mi-app.local hacia el Service
4	**Probar**	Entrar con curl usando la cabecera Host, o editando el archivo hosts
5	**HPA**	Crear un HorizontalPodAutoscaler: mínimo 2, máximo 10, objetivo 50 % de CPU
6	**Ver escalar**	Generar carga y observar con kubectl get hpa --watch cómo suben las réplicas

Los pasos 1 y 2 son los que hacen fallar a todo el mundo, y los dos fallan sin dar ningún error.

Actividad 2 · Con qué arrancar

Dos archivos más y los comandos para probar que funcionan

ingress.yaml

```yaml
# ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress
spec:
  ingressClassName: nginx
  rules:
    - host: mi-app.local
      http:
        paths:
          - path: /
            pathType: Prefix
```

hpa.yaml

```yaml
# hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 2
  maxReplicas: 10
  metrics:
```

como probar que funciona

```
# Probar el Ingress sin tocar el archivo hosts:
curl -H "Host: mi-app.local" http://$(minikube ip)

# Generar carga y ver escalar (dejar corriendo en otra terminal):
kubectl run carga --rm -it --image=busybox -- /bin/sh -c "while true; do wget -q -O- http://web-svc; done"
kubectl get hpa --watch
```

Actividad 2 · Qué tienen que ver

Los tres comandos que confirman que funcionó

El Ingress tiene dirección asignada

La columna ADDRESS no puede estar vacía. Si lo está, falta el controller.

```
kubectl get ingress
```

El HPA lee métricas de verdad

La columna TARGETS tiene que mostrar un porcentaje, no <unknown>.

```
kubectl get hpa
```

Las réplicas suben con la carga

REPLICAS pasa de 2 a 3, 4... Tarda entre 1 y 2 minutos en reaccionar.

```
kubectl get hpa --watch
```

Si TARGETS dice <unknown>: falta metrics-server, o falta resources.requests.cpu en el Deployment.

Puesta en común

Preguntas para pensar más allá de que haya funcionado

1

¿Qué ventaja tiene el Ingress sobre un NodePort por servicio?

Una sola IP y un solo certificado TLS para todos los servicios, con las reglas en un solo lugar.

2

¿Qué pasa si el HPA llega al máximo de réplicas y la carga sigue subiendo?

Se queda ahí: el HPA no crea nodos. Eso lo hace el Cluster Autoscaler, que es otra cosa.

3

¿Escalar por CPU siempre es lo correcto?

No. Si tu cuello de botella es una cola de mensajes o la base de datos, vas a tener pods ociosos.

4

¿Por qué el HPA tarda en bajar las réplicas?

Para no prender y apagar todo el tiempo. Subir es rápido, bajar es conservador.

Las respuestas en cursiva son para vos: preguntá primero, dejá que el grupo intente, y recién después confirmá.

Lo que nos llevamos

Y cómo se conecta con lo que viene

Estado deseado	Declarás cómo tiene que verse el mundo y Kubernetes lo sostiene. De ahí sale todo lo demás.
Deployment y Service	Uno mantiene los pods vivos, el otro les da una dirección estable. Se encuentran por labels.
Ingress	Una sola puerta de entrada HTTP que enruta por dominio o ruta hacia cada Service.
HPA	Ajusta réplicas según carga, pero necesita requests y metrics-server o no hace nada.

Próxima clase — Módulo 5: Terraform. El mismo modelo declarativo de hoy, pero para crear el cluster entero.

Para la próxima clase

Dos consignas sobre el repositorio que venís armando

1 Desplegar tu propia app

- Usar la imagen que construiste en el módulo 3, no nginx
- Deployment con 3 réplicas, Service, Ingress y HPA
- Agregar readiness y liveness probes
- Capturar el HPA escalando con carga real

2 Probar la resiliencia

- Hacer un rolling update a v2 con tráfico corriendo
- Documentar si hubo requests fallidas y por qué
- Repetirlo con replicas: 1 y comparar el resultado

La consigna 2 con replicas: 1 es la más formativa de las dos

Van a ver requests fallidas con sus propios ojos, y ahí entiende cualquiera por qué la alta disponibilidad necesita más de una réplica.

PREGUNTAS Y RESPUESTAS · 10 MINUTOS

¿Qué quedó dando vueltas?

Si nadie arranca, estas son las que siempre aparecen:

1. ¿Necesito Kubernetes para mi proyecto, o es demasiado?
2. ¿Cómo paso de Minikube a un cluster real en la nube?
3. ¿Dónde guardo la base de datos si los pods son descartables?
4. ¿Cómo conecto esto con el pipeline de GitHub Actions del módulo 2?
5. ¿Qué pasa si se cae el nodo entero, no solo un pod?

Las respuestas desarrolladas están en el guión docente.

MATERIAL DE APOYO

Anexo

Lo que no entra en la sesión en vivo pero sí en el módulo y en la evaluación.

Anexo · Probes y recursos

Lo que hace falta para que un despliegue sea realmente sin corte

readiness

¿Puede recibir tráfico ahora?

Si falla, el Service deja de mandarle tráfico pero el pod sigue vivo. Sin esto no hay rolling update sin corte.

liveness

¿Sigue vivo o se colgó?

Si falla, Kubernetes mata el contenedor y lo reinicia. Si la ponés muy agresiva, entrás en un bucle de reinicios.

```
resources:
  requests:          # lo que tengo garantizado
    memory: "64Mi"   # Mi = mebibyte (1024 x 1024)
    cpu: "100m"      # m = millicores, 1000m = 1 nucleo
  limits:            # el techo
    memory: "256Mi"
```

La asimetría que importa

Si pasás el límite de CPU, te estrangulan: la app va más lenta pero sigue viva. Si pasás el de memoria, Kubernetes mata el contenedor. En kubectl aparece como OOMKilled.

Un pod que se reinicia solo cada tanto sin error en los logs es, casi siempre, un limit de memoria demasiado bajo.

Anexo · Operar un cluster en producción

Las cinco tareas del módulo que la sesión práctica no alcanza a cubrir

Provisionamiento	Administrado (EKS, GKE, AKS) o autogestionado. Si no tenés un equipo dedicado, administrado.
Upgrades	Primero el control plane, después los nodos de a uno con cordon y drain. Una versión menor por vez.
Backups	Snapshot de etcd, que guarda TODO el estado. Más tus manifiestos en Git. Más los volúmenes aparte.
Gestión de nodos	Agregar y sacar nodos sin cortar servicio. Agruparlos por tipo según la carga que corren.
Capacidad	Sumar los requests de todo y dejar margen para perder un nodo entero sin quedarte corto.

Si perdés etcd, perdiste el cluster aunque los nodos estén intactos. Y un backup que nunca restauraste no es un backup.